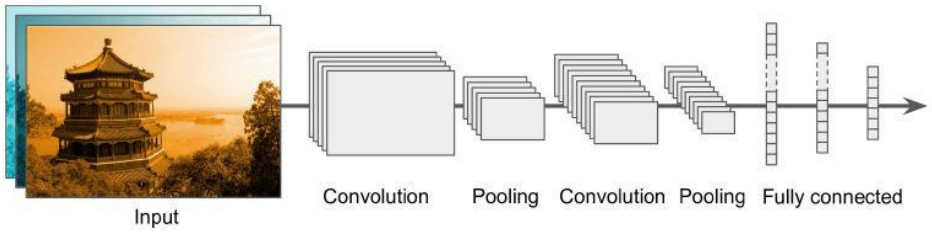
Week 8 – ITA - ML.

Machine Learning

21 Oct 2026

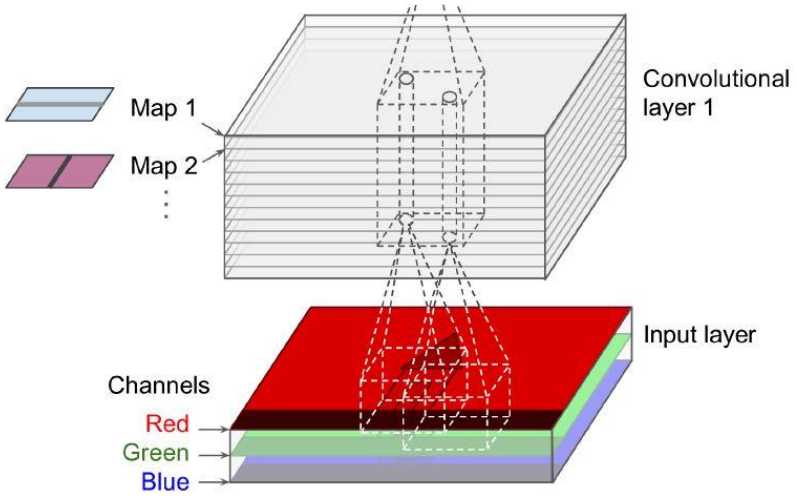


Hands on Machine Learning. Chapter. 14. Aurelien Geron.

Sila Oct 2026.

Today:
Convolutional Neural Networks

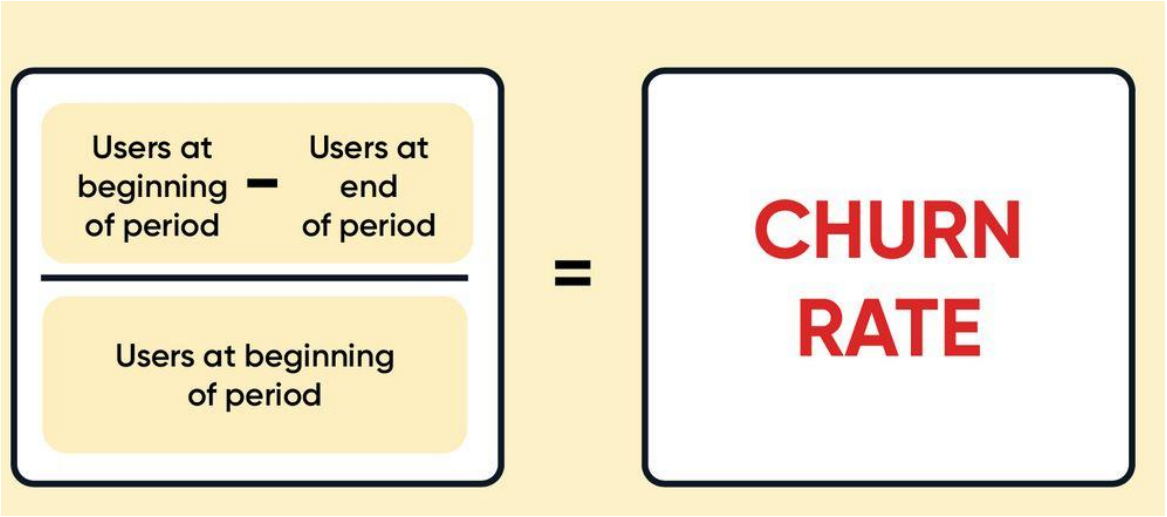
- Churn exercise.
- Activation functions.
- Batch normalizations.
- Convolutions.



Hands on Machine Learning. Chapter. 14. Aurelien Geron.

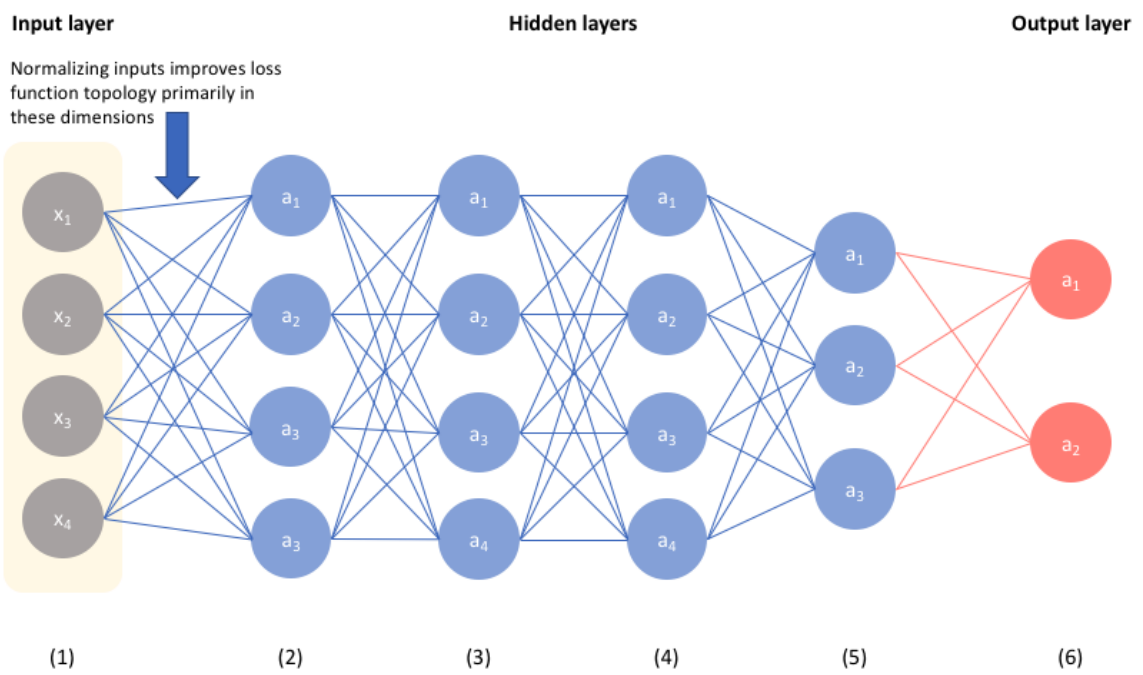
Excercise 1.

Churn Exercise.
See Canvas.

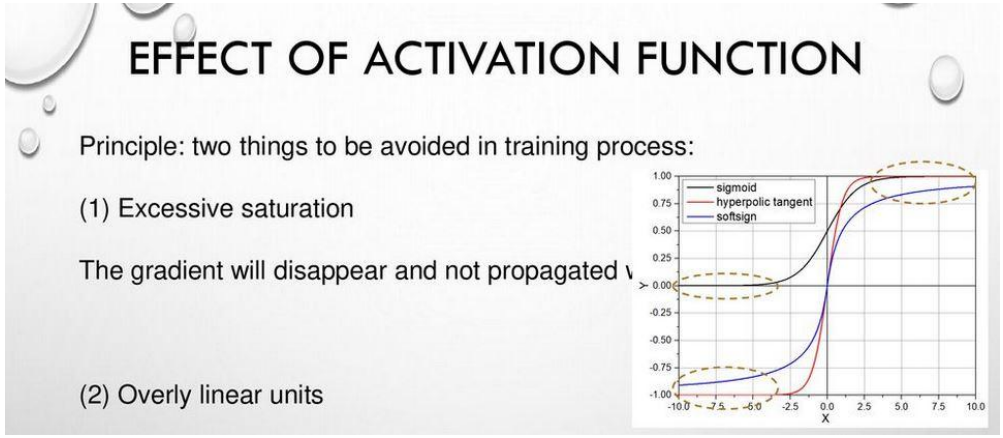


Deep Neural Nets.

Today:
A closer look.
Activation functions and more.



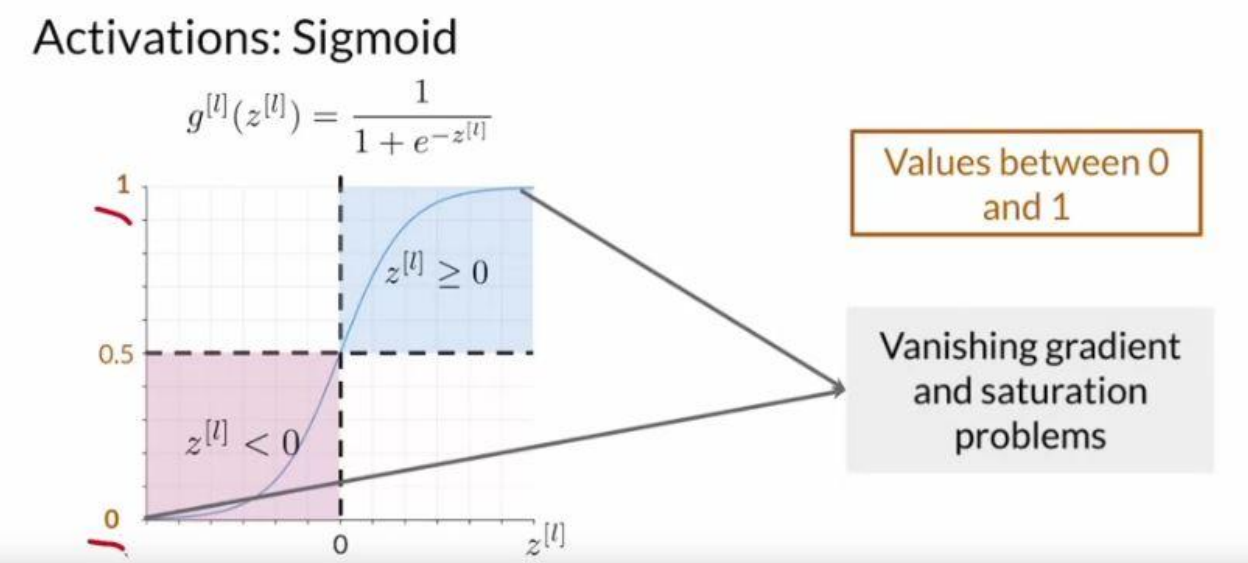
Recap:
Activation Functions (Neural Nets)



Source: http://neuralnetworksanddeeplearning.com/chap5.html

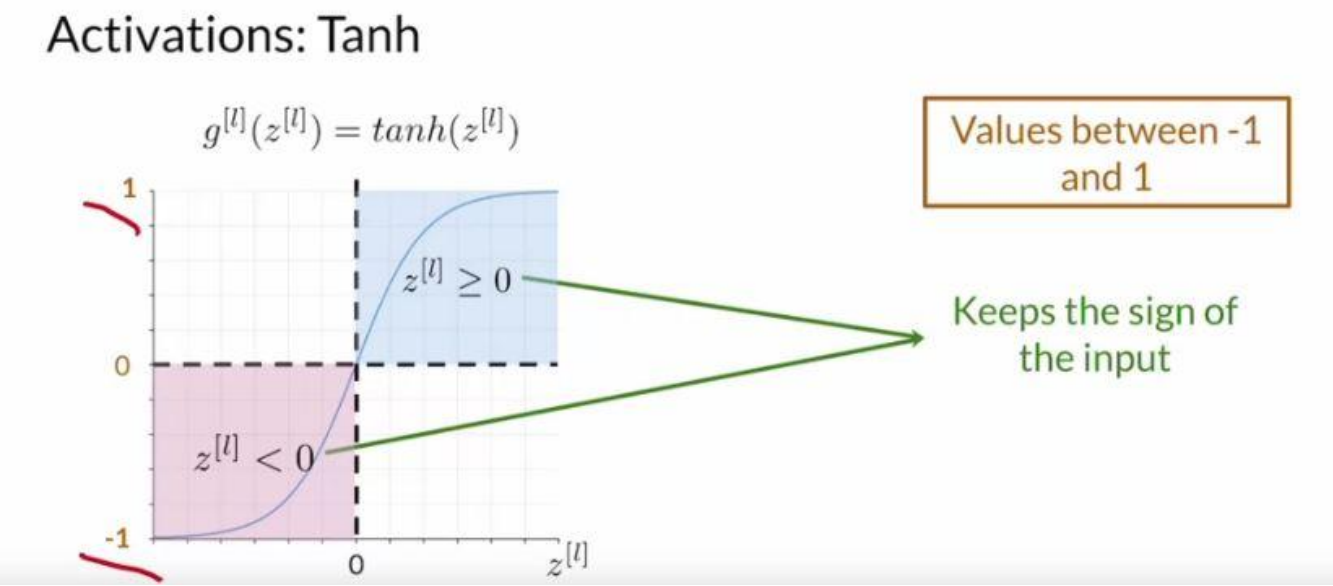
We have here an important observation: in at least some deep neural networks, ***the gradient tends to get smaller as we move backward through the hidden layers. This means that neurons in the earlier layers learn much more slowly than neurons in later layers***. And while we've seen this in just a single network, there are fundamental reasons why this happens in many neural networks. The phenomenon is known as the *vanishing gradient problem*

Recap:
Activation Functions (Neural Nets)



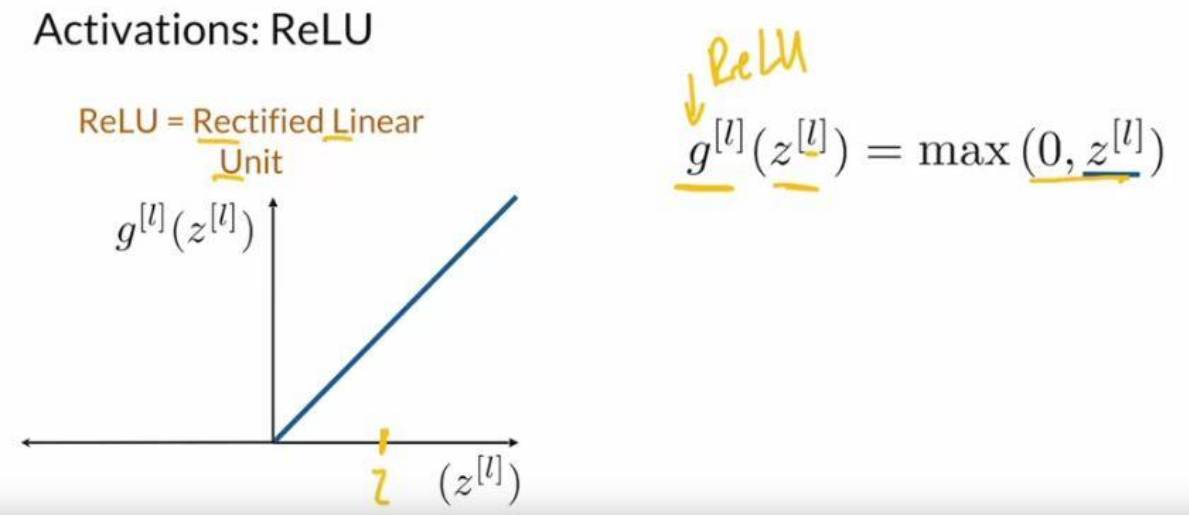
Source: Coursera.

Recap:
Activation Functions (Neural Nets)



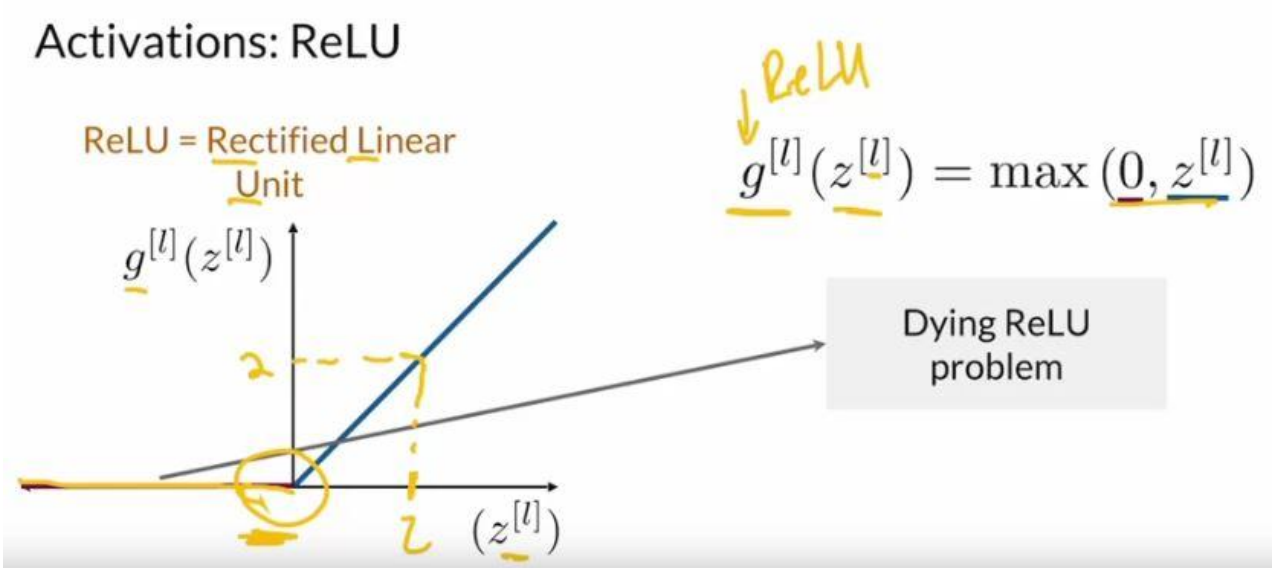
Source: Coursera.

Recap:
Activation Functions (Neural Nets)



Source: Coursera.

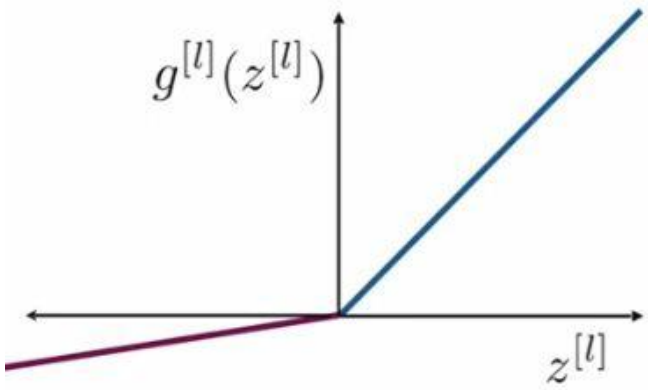
Recap:
Activation Functions (Neural Nets)



Source: Coursera.

Recap:
Activation Functions (Neural Nets)

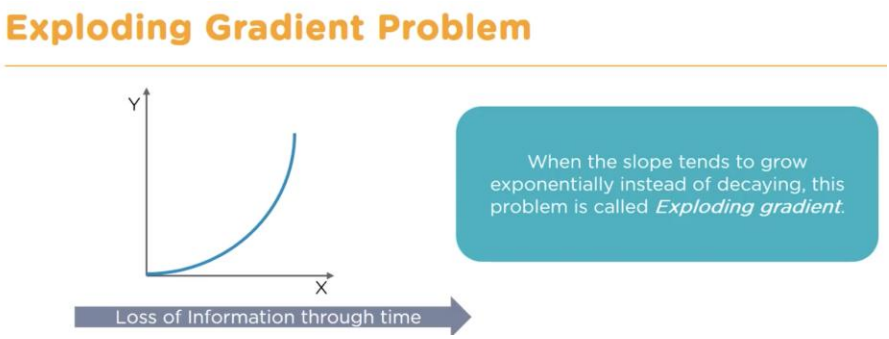
Activations: Leaky ReLU



$$g^{[l]}(z^{[l]}) = max(az^{[l]}, z^{[l]})$$

Source: Coursera.

Recap:
Exploding gradients (Neural Nets)



Source: http://neuralnetworksanddeeplearning.com/chap5.html

The exploding gradient problem: Let's look at an explicit example where exploding gradients occur. The example is somewhat contrived: I'm going to fix parameters in the network in just the right way to ensure we get an exploding gradient. But even though the example is contrived, it has the virtue of firmly establishing that exploding gradients aren't merely a hypothetical possibility, they really can happen.

Recap:
Exploding gradients, vanishing gradients

Let chatGPT explain it to you.

Why does it matter what the activation function is?

If the explanation is difficult to understand, then ask ChatGPT to make the explanation simpler.

Batch Normalization

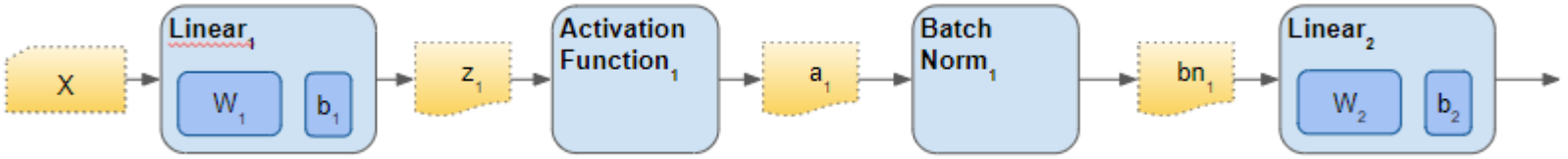
Batch normalization (also known as batch norm) is a method used to make artificial neural networks faster and more stable through normalization of the layers inputs by re-centering and re-scaling.

Using BatchNorm, we add a normalization step that fixes the means and variances of layer inputs which helps in faster convergence and improved gradient flow through the network, by reducing the dependence of gradients on the scale of the parameters or of their initial values.

Source: https://towardsdatascience.com/batchnormalization-is-not-a-norm-cebfd921dadd

Batch Normalization

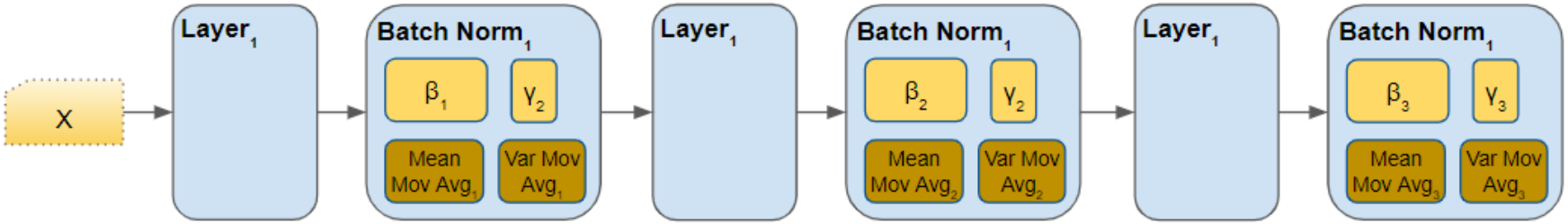
Batch normalization (also known as batch norm) is a method used to make artificial neural networks faster and more stable through normalization of the layers inputs by re-centering and re-scaling.



Source: https://towardsdatascience.com/batchnormalization-is-not-a-norm-cebfd921dadd

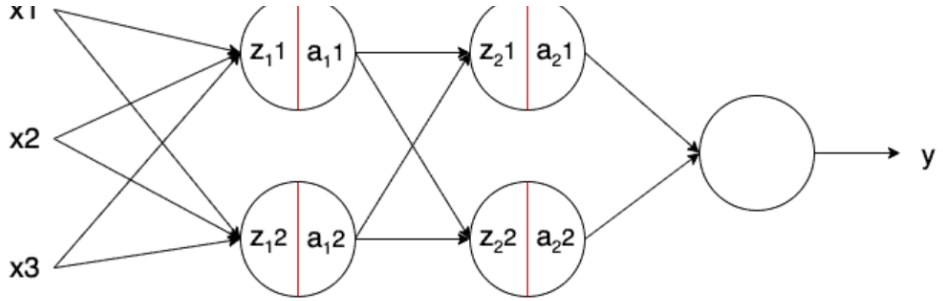
Batch Normalization

Batch normalization (also known as batch norm) is a method used to make artificial neural networks faster and more stable through normalization of the layers inputs by re-centering and re-scaling.



Source: https://towardsdatascience.com/batchnormalization-is-not-a-norm-cebfd921dadd

Batch Normalization.



Batch Norm – in the image represented with a red line – is applied to the neurons' output just before applying the activation function. Usually, a neuron without Batch Norm would be computed as follows:

$$z = g(w, x) + b; \qquad a = f(z)$$

being $g()$ the linear transformation of the neuron, w the weights of the neuron, b the bias of the neurons, and $f()$ the activation function. The model learns the parameters w and b. Adding Batch Norm, it looks as:

$$z = g(w, x); \qquad z^N = \left(\frac{z - m_z}{s_z}\right) \cdot \gamma + \beta; \qquad a = f(z^N)$$

https://www.baeldung.com/cs/batch-normalization-cnn

Batch Normalization.

1. Normalisering af Minibatches

Batch normalization anvendes på outputtet af et givent lag X i netværket. Hvert element X_i i minibatch $X = \{x_1, x_2, ..., x_m\}$ normaliseres baseret på batchens middelværdi og varians:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

Her er μ_B middelværdien og σ_B^2 variansen for minibatchen X.

Batch Normalization.

2. Standardisering af Hver Input

Input standardiseres ved at trække middelværdien fra og dividere med standardafvigelsen:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Her bruges en lille værdi ϵ (f.eks. 10^{-5}) for at undgå division med nul.

3. Skalering og Forskydning

Efter normalisering anvendes en skaleringsparameter γ og en forskydningsparameter β, der begge læres under træningen:

$$y_i = \gamma \hat{x}_i + \beta$$

Denne transformation sikrer, at batch normalization ikke er begrænset til at producere data med middelværdi 0 og varians 1; i stedet kan det tilpasses den bedst mulige fordeling for modellen.

Batch Normalization. Ask chatGPT. Try yourself…

1. **Mini-batch Statistics Calculation**: For a given input batch to the layer, the algorithm calculates the mean and variance of the inputs for each feature. These statistics are computed across the mini-batch for each input independently.
2. **Normalization**: The inputs for each feature are then normalized by subtracting the mini-batch mean and dividing by the square root of the mini-batch variance plus a small epsilon (to prevent division by zero). This step ensures that the inputs to the layer have a mean of 0 and a variance of 1.
3. **Scaling and Shifting**: After normalization, the outputs are then scaled and shifted using two trainable parameters, gamma (scaling) and beta (shifting). This step allows the network to undo the normalization if it determines that it is not beneficial for that layer. The scaling and shifting parameters are learned along with the original model parameters during training.

$$\mathrm{BN}(x) = \gamma \left(\frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} \right) + \beta$$

where:

- x is the input to the batch normalization layer,
- μ is the mini-batch mean,
- σ^2 is the mini-batch variance,
- ϵ is a small constant added for numeric[illegible]stability,
- γ and β are parameters to be learned during training.

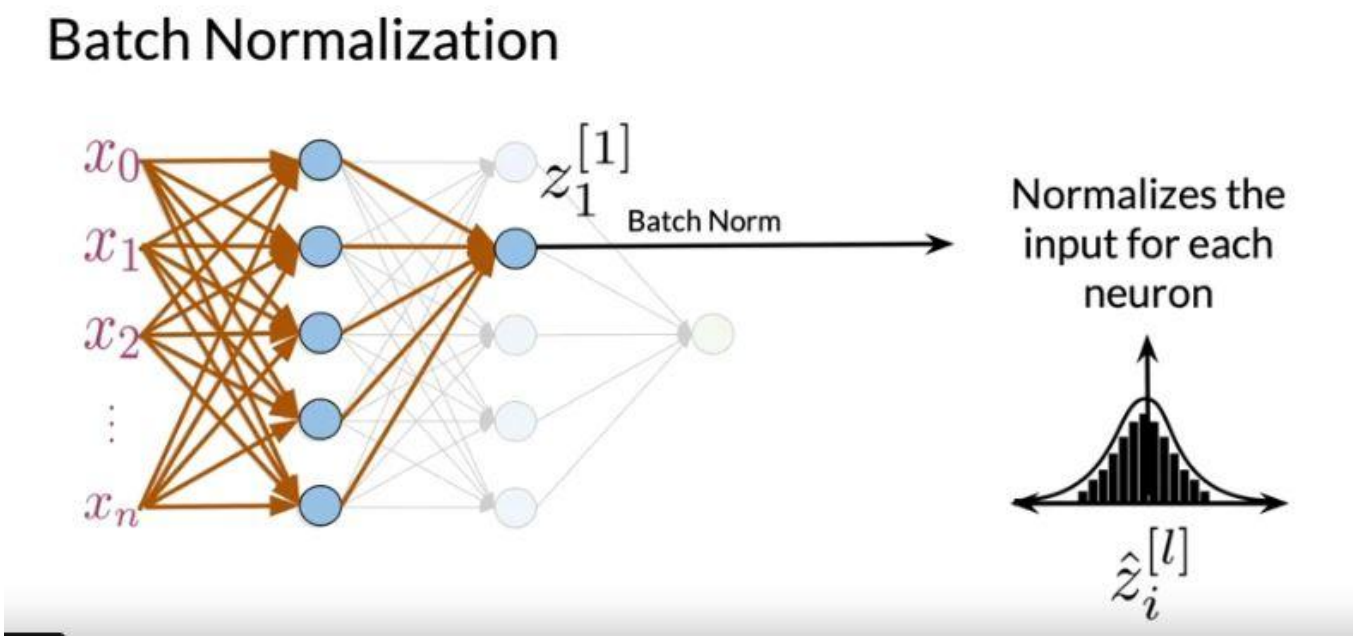
Batch Normalization – Benefits.

Higher learning rate.

If we use a high learning rate in a traditional neural network, then the gradients could explode or vanish. Large learning rates can scale the parameters which could amplify the gradients, thus leading to an explosion. But if we do batch normalization, small changes in parameter to one layer do not get propagated to other layers. This makes it possible to use higher learning rates for the optimizers, which otherwise would not have been possible. It also makes gradient propagation in the network more stable.

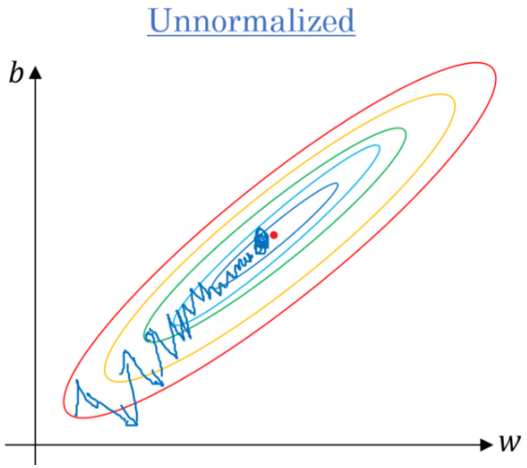
Source: https://learnopencv.com/batch-normalization-in-deep-networks/

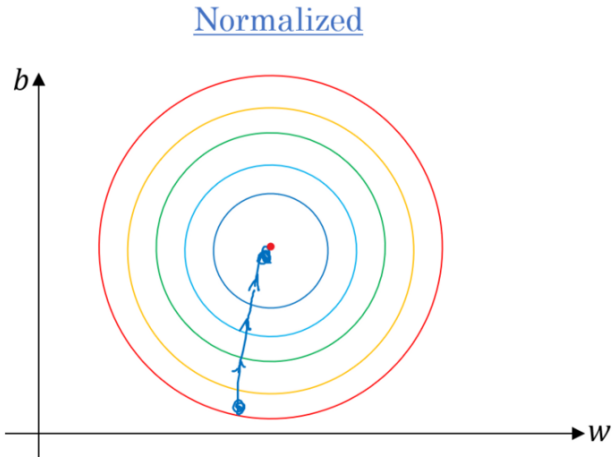
Batch Normalization

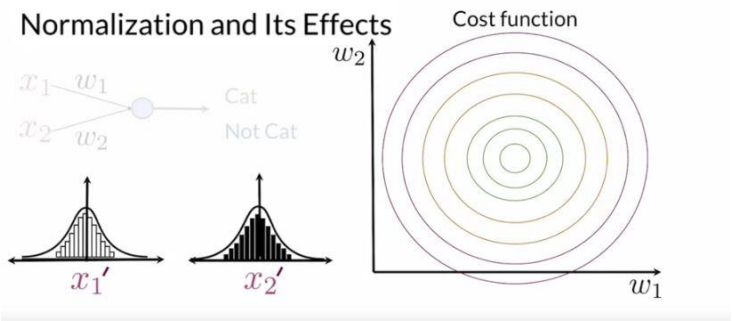


Source: Coursera.

Batch Normalization

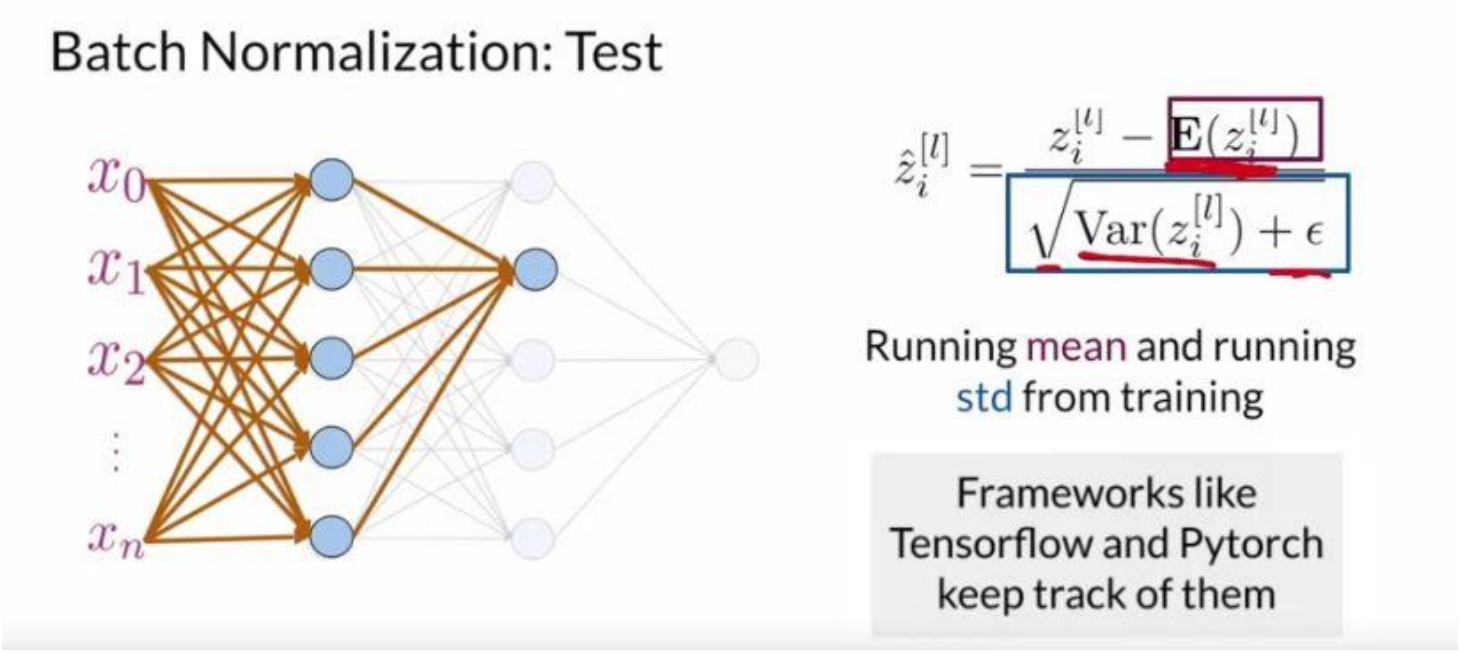






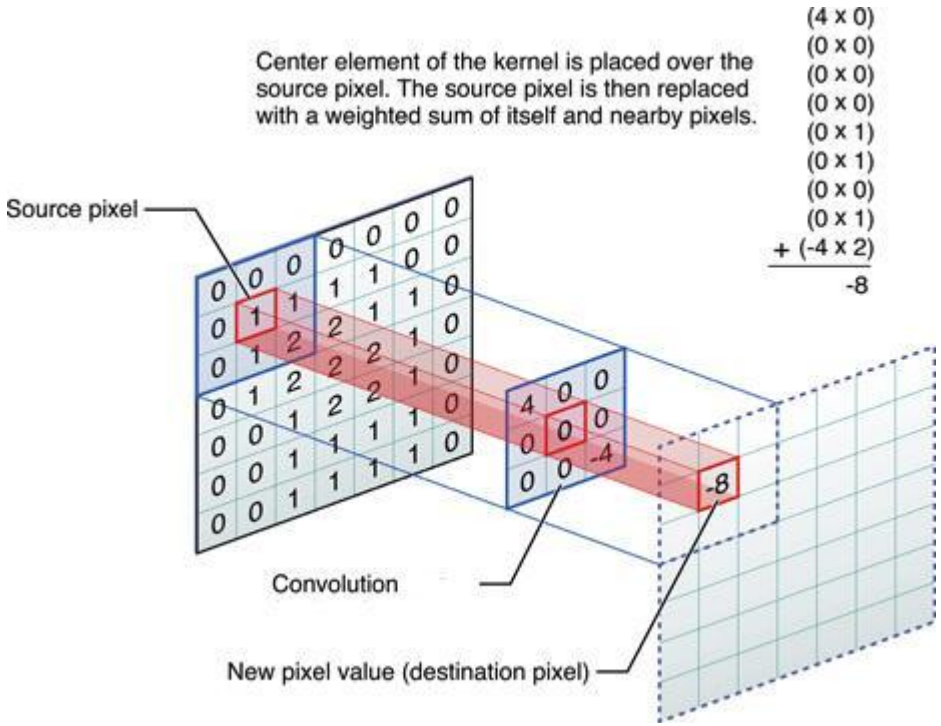
Source: Coursera.

Batch Normalization



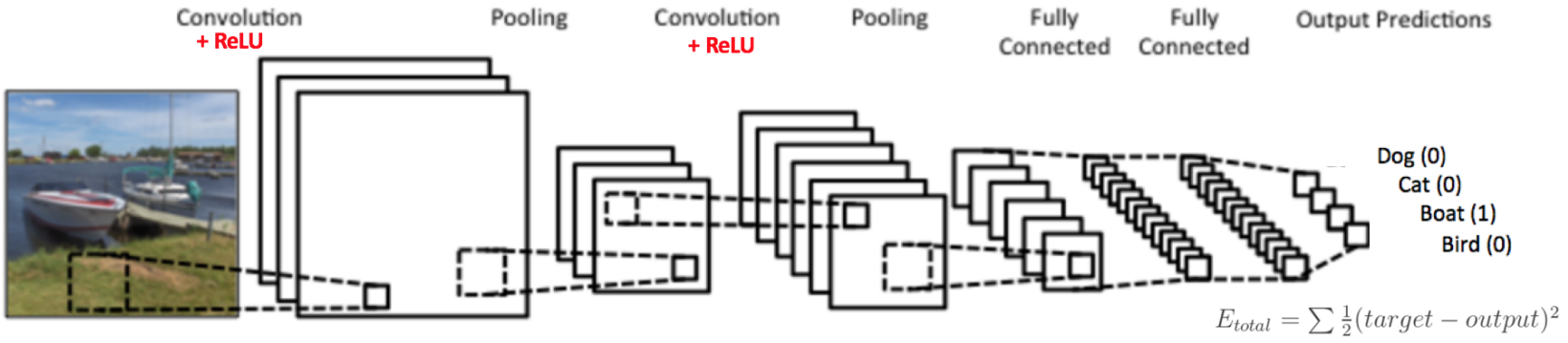
Source: Coursera.

Convolutions



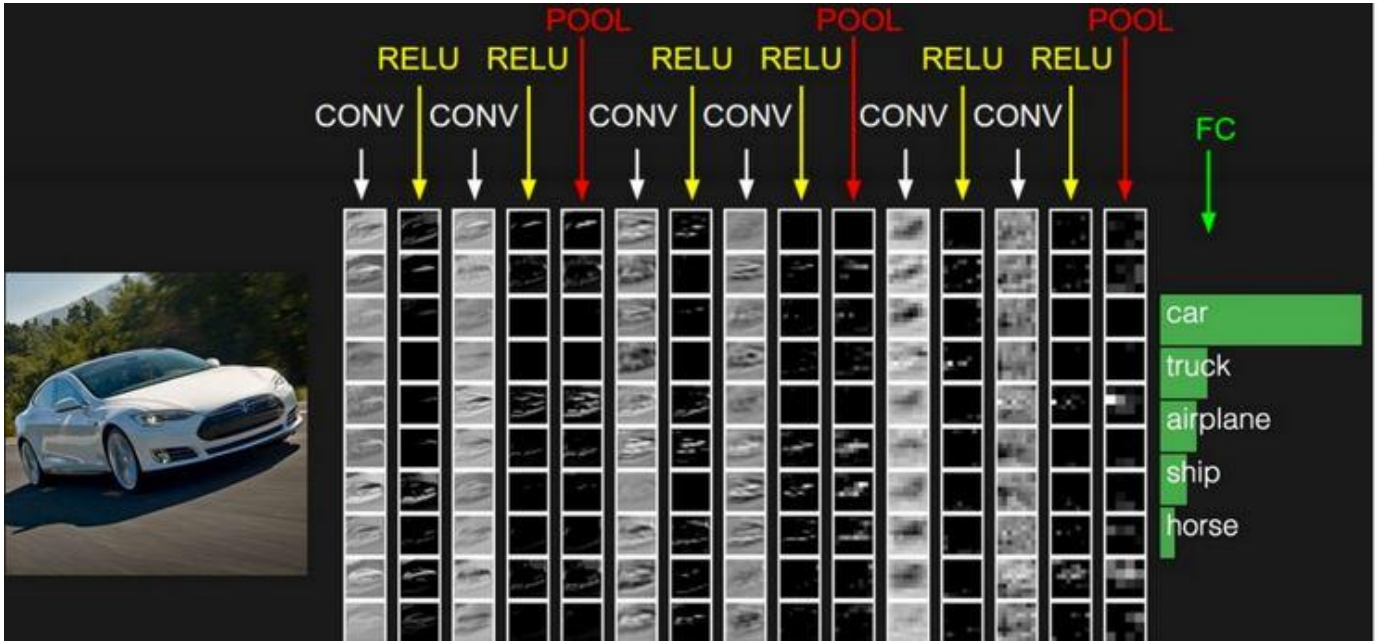
See: https://viniciuscantocosta.medium.com/understanding-the-structure-of-a-cnn-b220148e2ac4

Convolutions – Intuition.



See: https://www.kdnuggets.com/2016/11/intuitive-explanation-convolutional-neural-networks.html/3

Convolutions - Intuition.

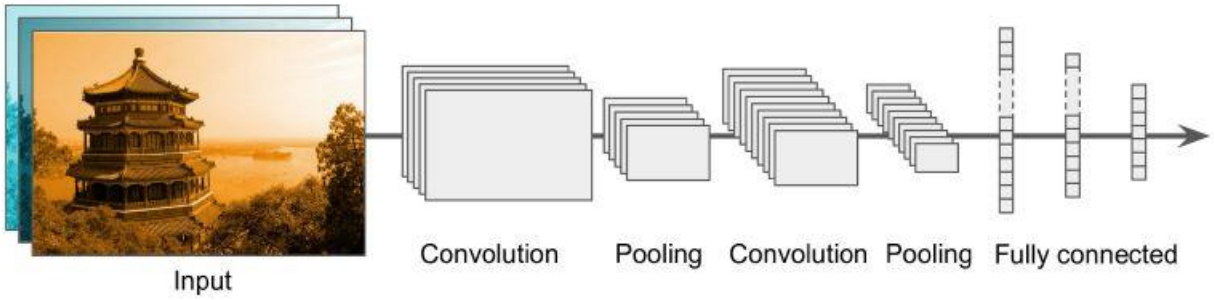


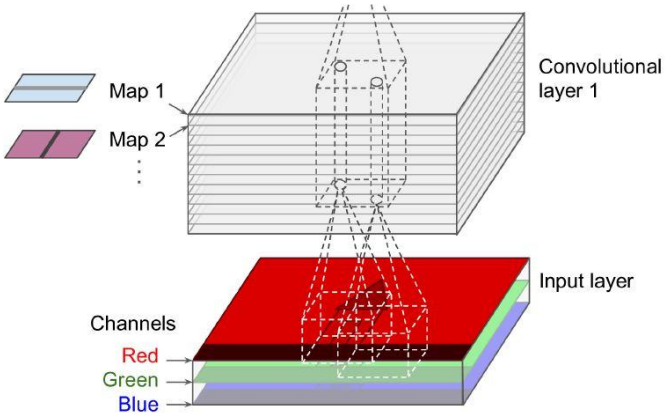
See: https://www.kdnuggets.com/2016/11/intuitive-explanation-convolutional-neural-networks.html/3

Convolutions

- Convolutions are useful for processing images.
- The scan the image to detect useful features.
- Just element-wise products and sums.

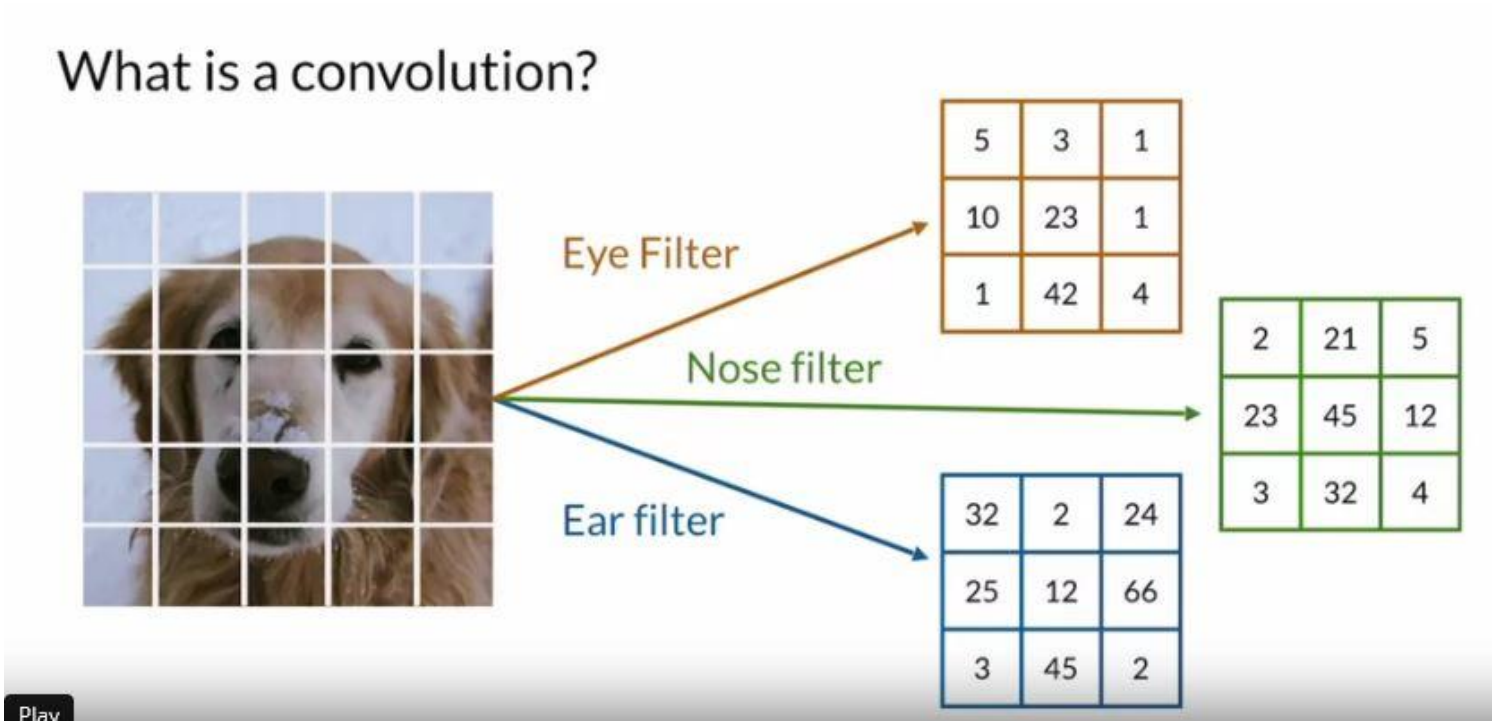
Convolutional Neural Networks





Hands on Machine Learning. Chapter. 14. Aurelien Geron.

Convolutions



Source: Coursera.

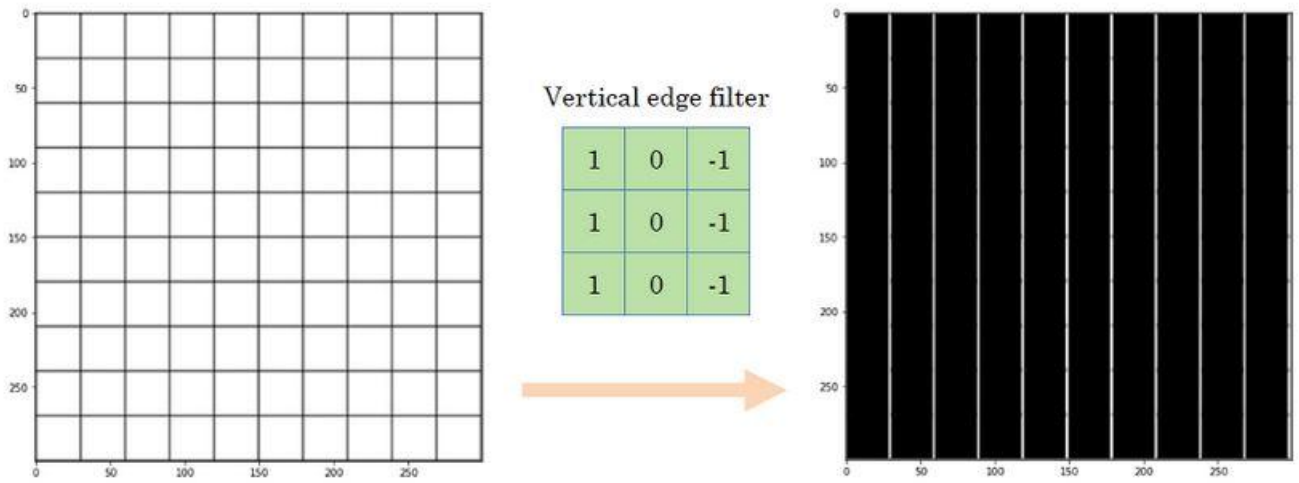
Convolutions – What it does.

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

Further explanation, see: https://towardsdatascience.com/understanding-convolutions-and-pooling-in-neural-networks-a-simple-explanation-885a2d78f211

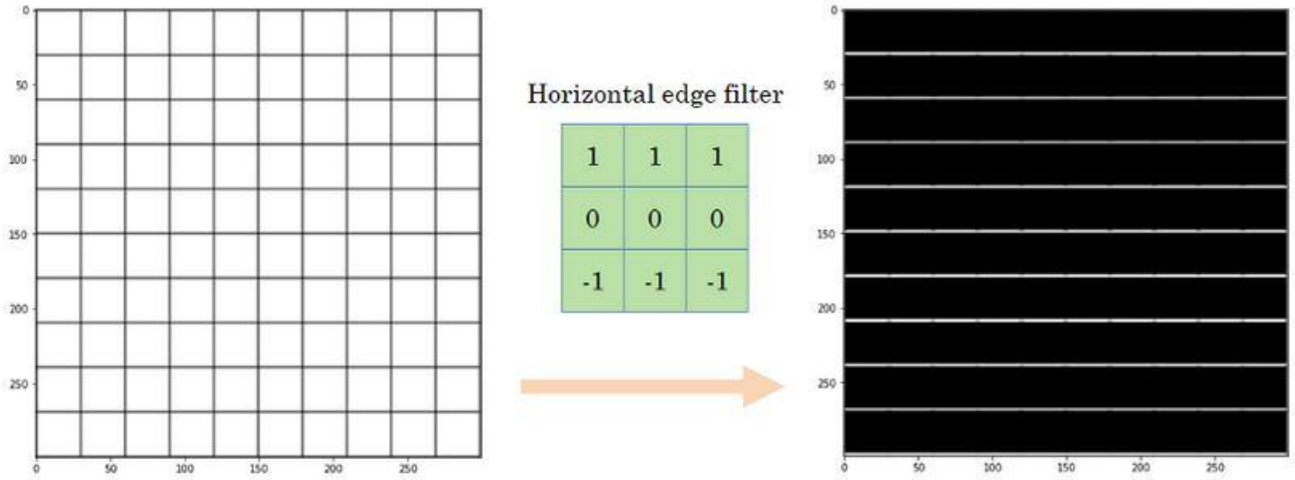
Convolutions



With this particular filter, the output image highlights the vertical edges.

Further explanation, see: https://towardsdatascience.com/understanding-convolutions-and-pooling-in-neural-networks-a-simple-explanation-885a2d78f211

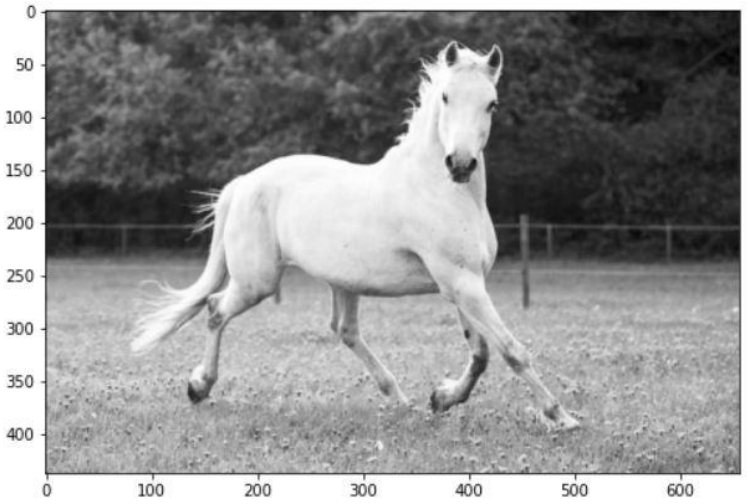
Convolutions

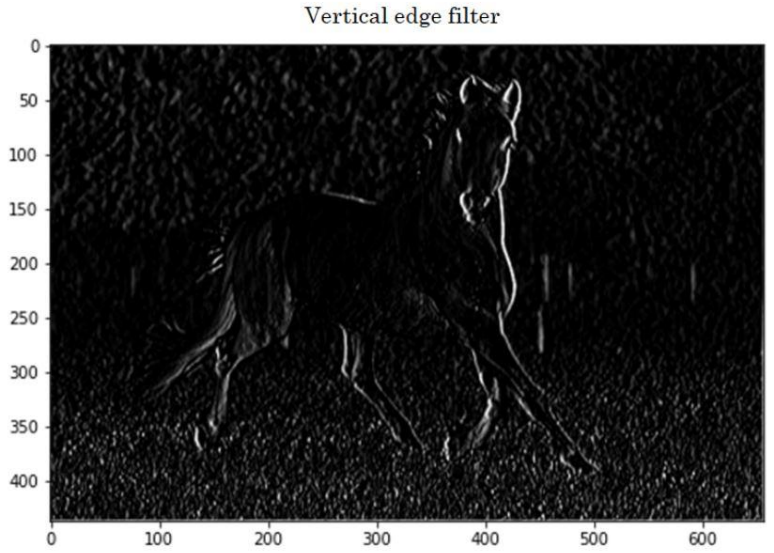


With this particular filter, the output image highlights the horisontal edges.

Further explanation, see: https://towardsdatascience.com/understanding-convolutions-and-pooling-in-neural-networks-a-simple-explanation-885a2d78f211

Convolutions vertical filter.

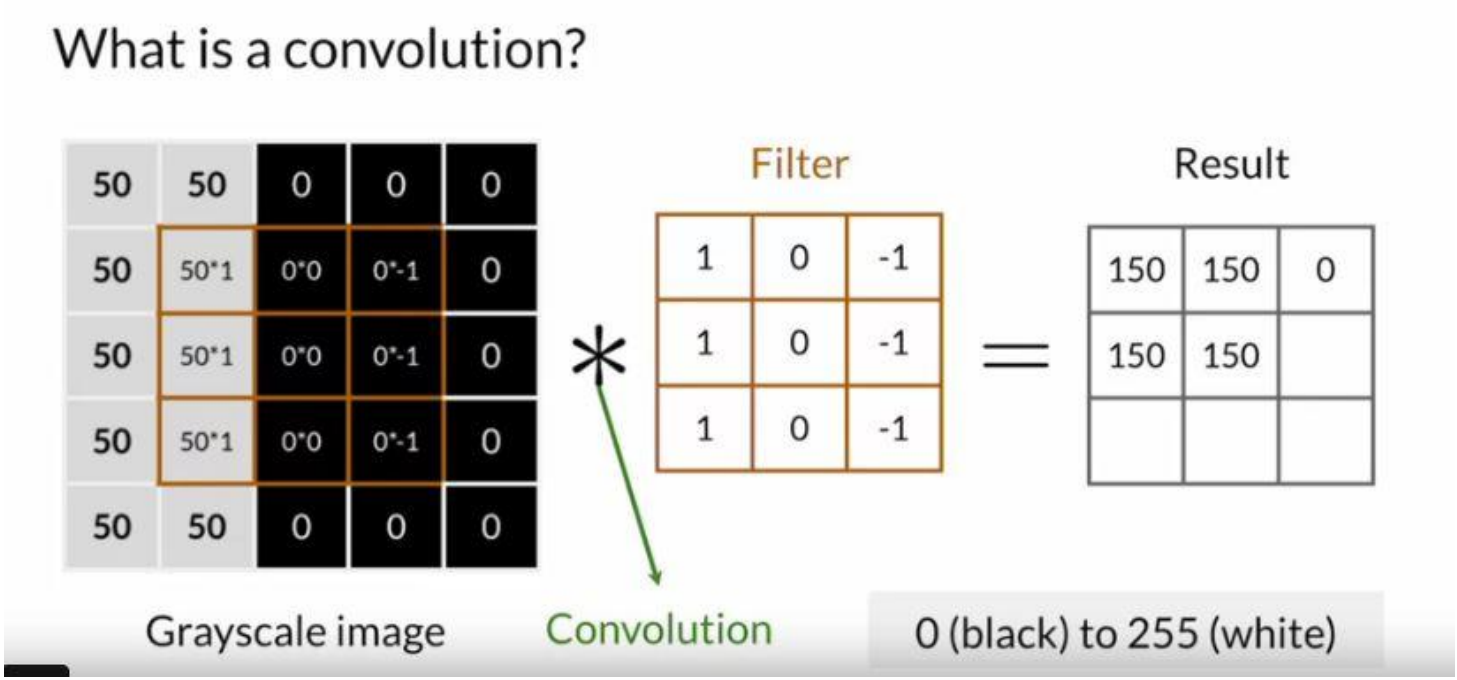




Vertical edge filter.

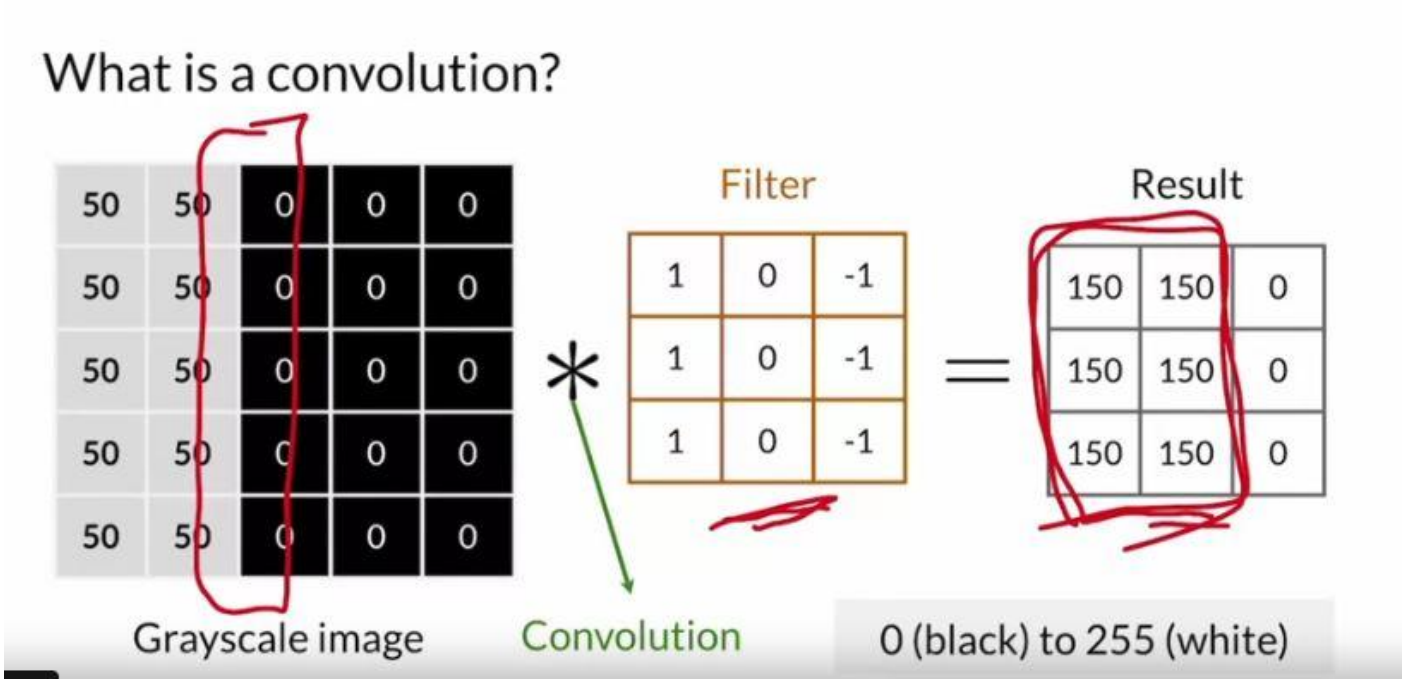
Further explanation, see: https://towardsdatascience.com/understanding-convolutions-and-pooling-in-neural-networks-a-simple-explanation-885a2d78f211

Convolutions



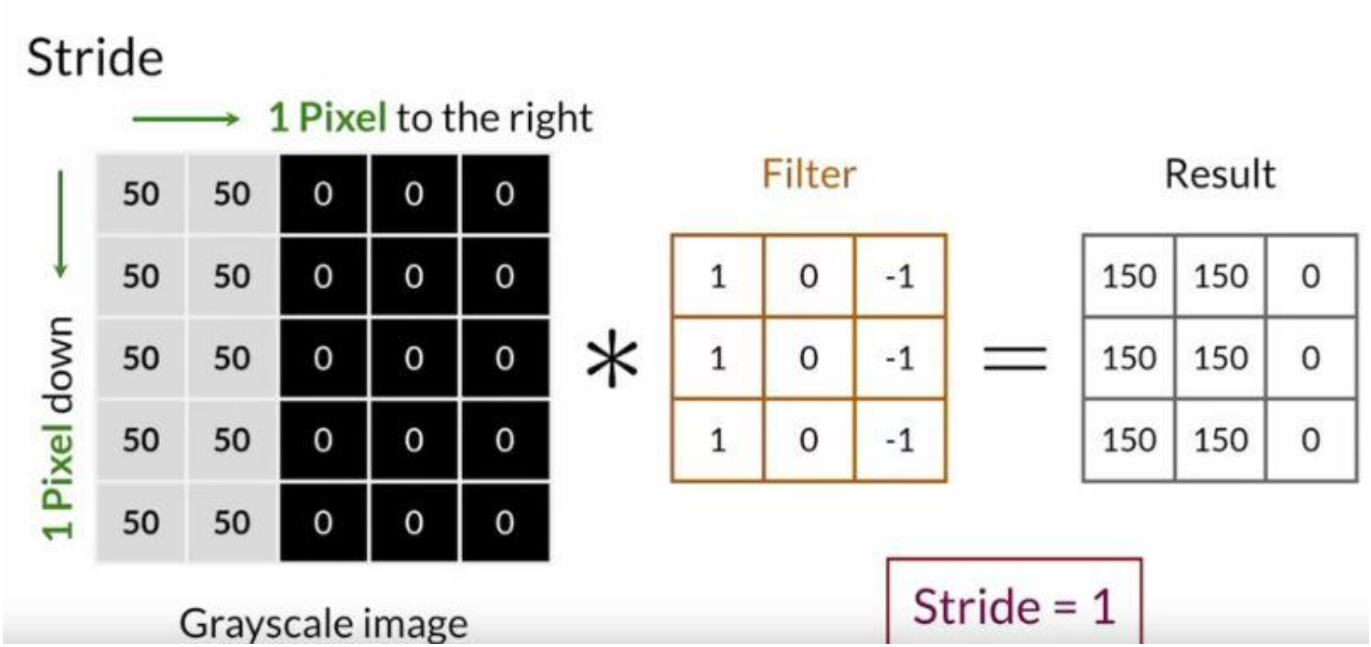
Further explanation, see: https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529

Convolutions



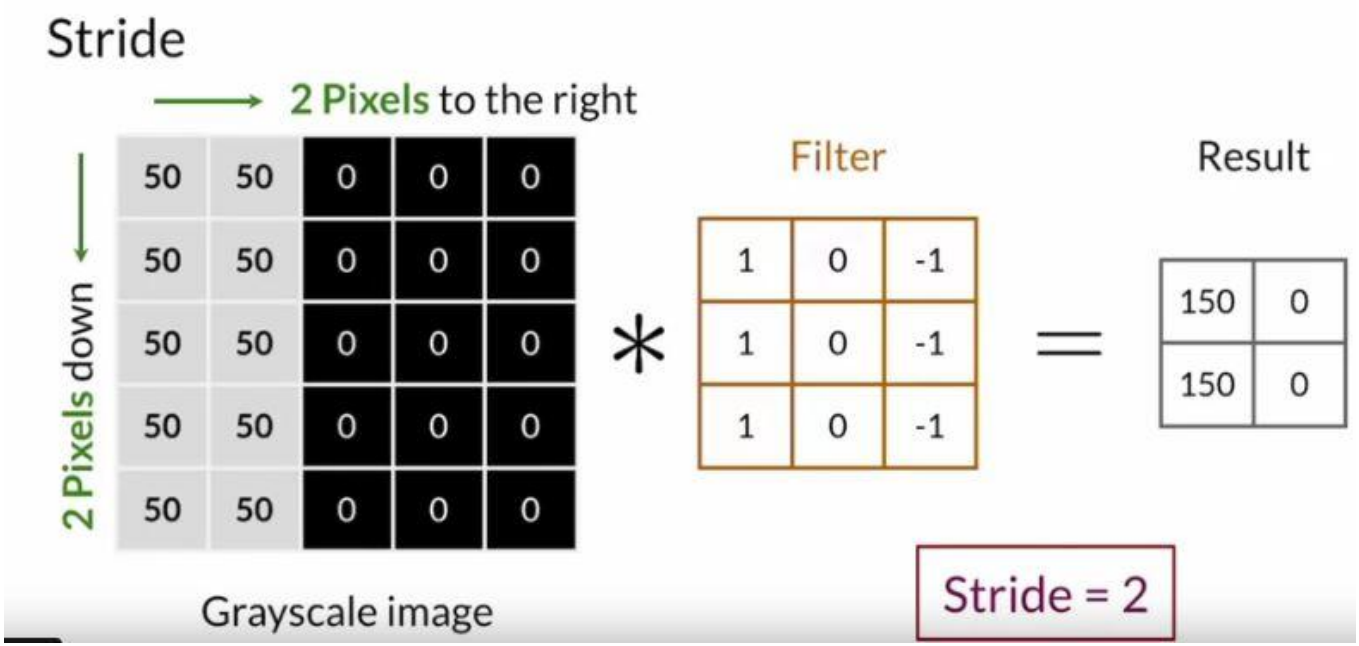
Further explanation, see: https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529

Convolutions - Stride.



Further explanation, see: https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529

Convolutions - Stride.

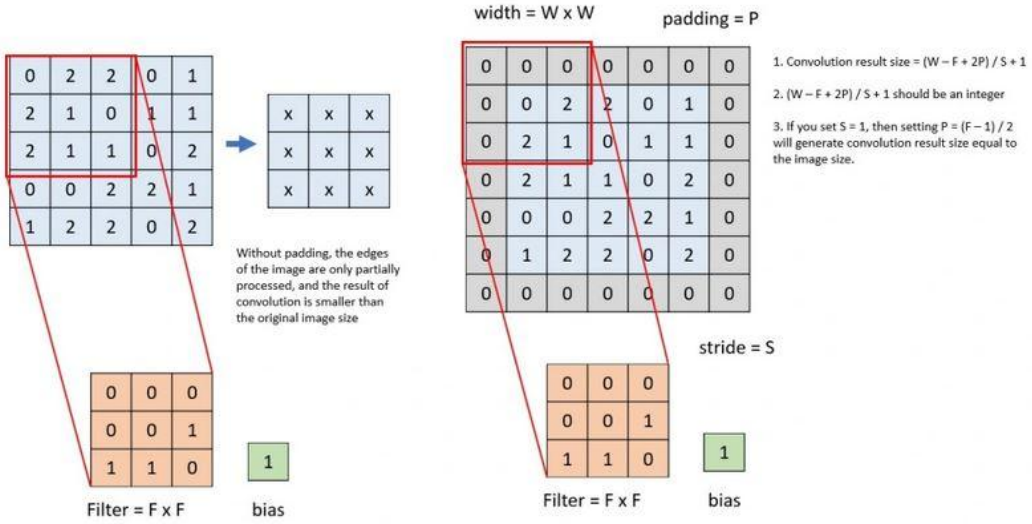


Further explanation, see: https://towardsdatascience.com/covolutional-neural-network-cb0883dd6529

Stride and Padding.

- Stride determines how the filter scans the image.
- Padding is like a frame on the image.
- Padding gives similar importance to the edges and the center.

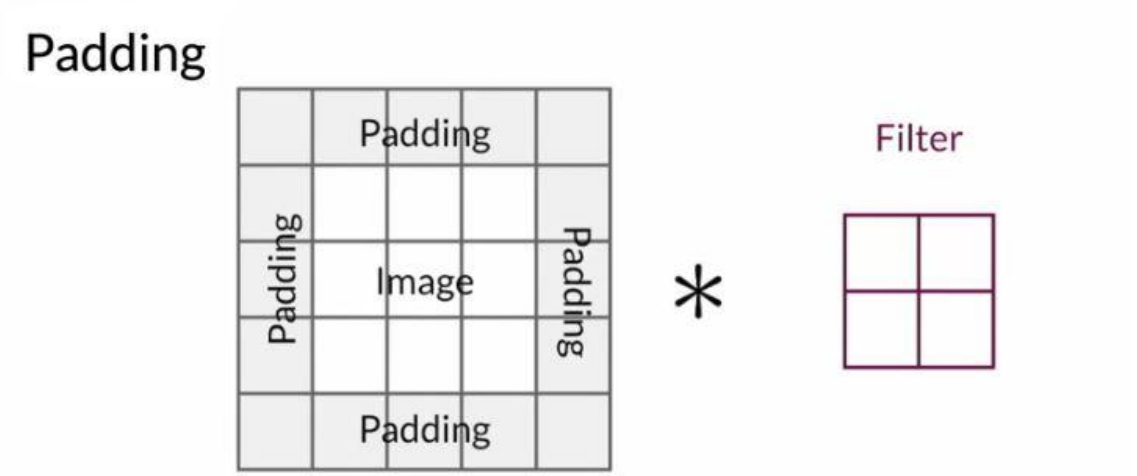
Image Padding.



Without image padding, the pixels on the edge of the image are only partially processed (which may be OK), and the result of convolution will be smaller than the original image size (usually not good).

Source: https://jamesmccaffrey.wordpress.com/2018/05/30/convolution-image-size-filter-size-padding-and-stride/

Image Padding.



Without image padding, the pixels on the edge of the image are only partially processed (which may be OK), and the result of convolution will be smaller than the original image size (usually not good).

Source: https://jamesmccaffrey.wordpress.com/2018/05/30/convolution-image-size-filter-size-padding-and-stride/

Exercise 2 & 3.

Exercise 2.
Getting started with convolutional layers.
Use a single convolutional layer to detect a line in some input data.
See the ML_Week_11_Exercise2_Convolution.py for a demo of the calculations.

Exercise 3.
In this exercise you should run the example from the book p. 492 (v2 p. 453)
See explanation there (ML_Week_11_Exercise3_Convolution.py).

Change the strides parameter to 10. And see what happens.
Look at output for both pictures.

Stacked Convolutional layers

```
# example of stacked convolutional layers
from keras.models import Sequential
from keras.layers import Conv2D
# create model
model = Sequential()
model.add(Conv2D(1, (3,3), input_shape=(8, 8, 1)))
model.add(Conv2D(1, (3,3)))
# summarize model
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_1 (Conv2D)            (None, 6, 6, 1)           10
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 4, 4, 1)           10
=================================================================
Total params: 20
Trainable params: 20
Non-trainable params: 0
```

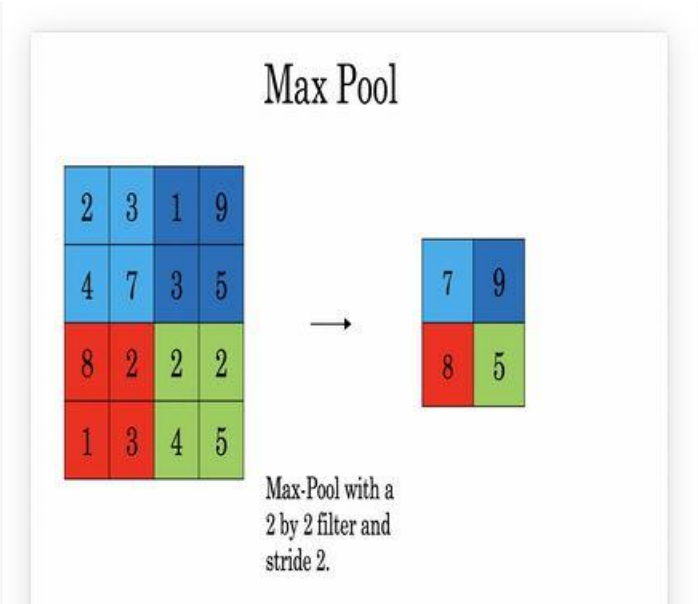
Running the example summarizes the shape of the output from each layer.

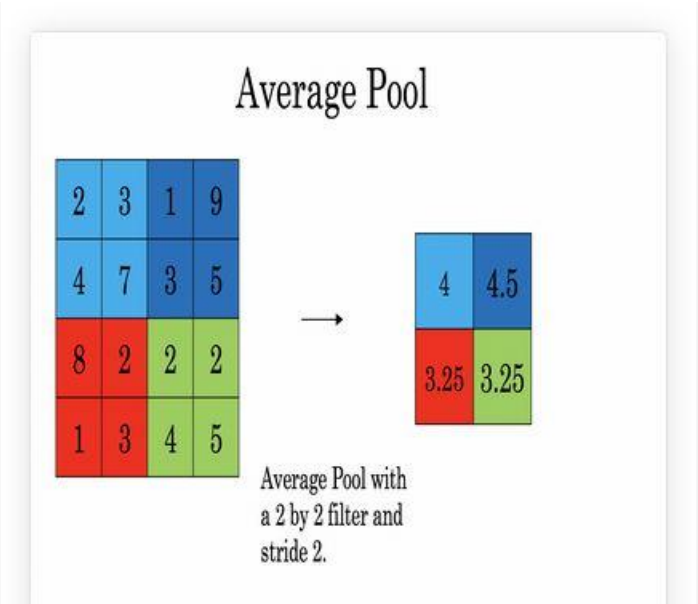
We can see that the application of filters to the feature map output of the first layer, in turn, results in a smaller 4×4 feature map.

This can become a problem as we develop very deep convolutional neural network models with tens or hundreds of layers.

We will simply run out of data in our feature maps upon which to operate.

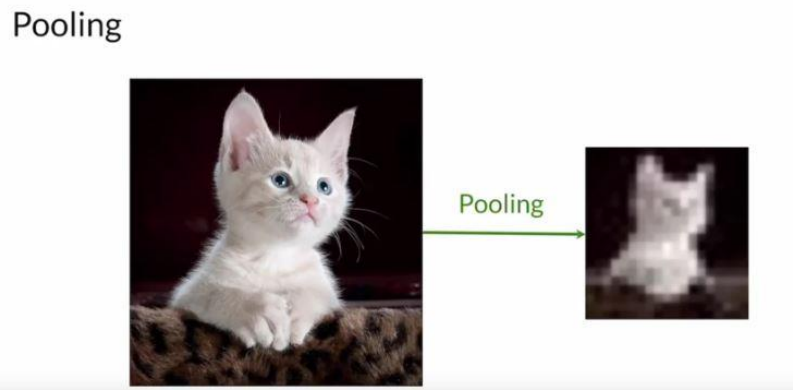
Pooling.





Source: https://www.cnblogs.com/geekfx/p/14212337.html

Pooling.

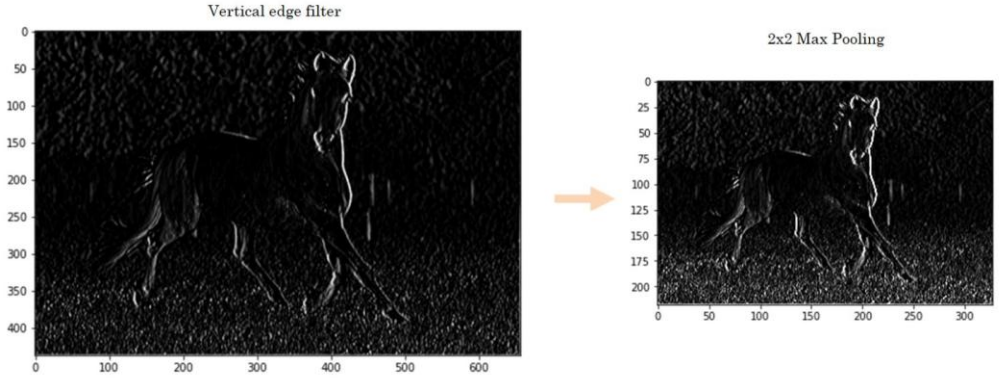


The image size is reduced to its half: by taking groups of 2x2 pixels and only retaining the maximum, now the image is half the size.

The edges are maintained.

Source: https://towardsdatascience.com/understanding-convolutions-and-pooling-in-neural-networks-a-simple-explanation-885a2d78f211

Pooling.



The image size is reduced to half: by taking groups of 2x2 pixels and only retaining the maximum, now the image is half the size.

The edges are maintained and even highlighted.

Source: https://towardsdatascience.com/understanding-convolutions-and-pooling-in-neural-networks-a-simple-explanation-885a2d78f211

Convolutional Neural Networks

AlexNet

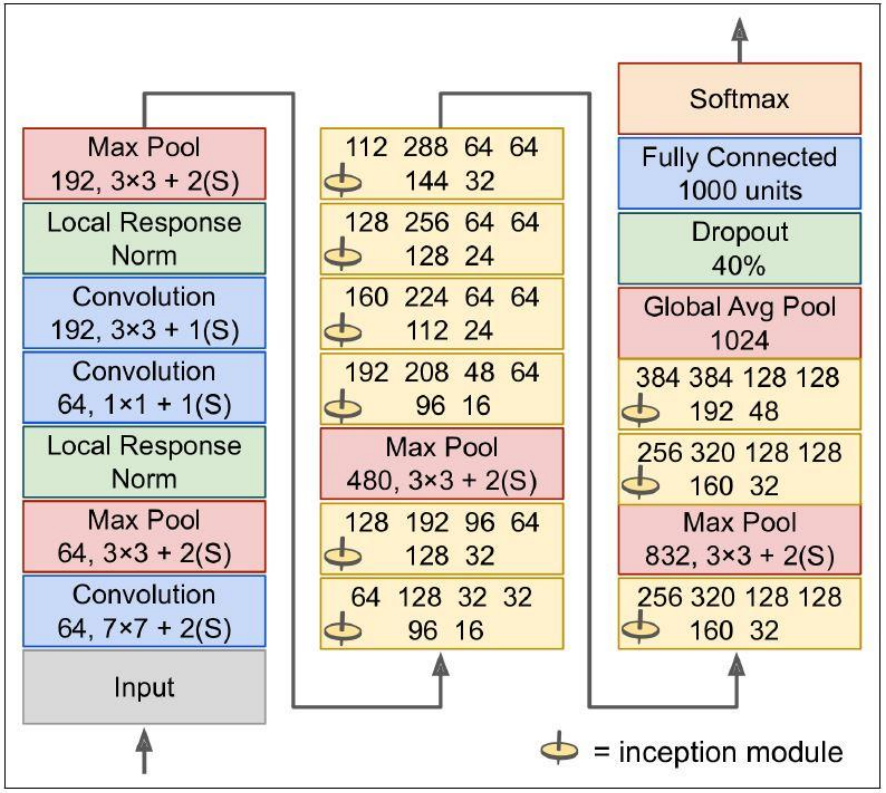
The *AlexNet* CNN architecture[11] won the 2012 ImageNet ILSVRC challenge by a large margin: it achieved 17% top-5 error rate while the second best achieved only 26%! It was developed by Alex Krizhevsky (hence the name), Ilya Sutskever, and Geoffrey Hinton. It is quite similar to LeNet-5, only much larger and deeper, and it was the first to stack convolutional layers directly on top of each other, instead of stacking a pooling layer on top of each convolutional layer. Table 14-2 presents this architecture.

Table 14-2. AlexNet architecture

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully Connected	–	1,000	–	–	–	Softmax
F9	Fully Connected	–	4,096	–	–	–	ReLU
F8	Fully Connected	–	4,096	–	–	–	ReLU
C7	Convolution	256	13 × 13	3 × 3	1	SAME	ReLU
C6	Convolution	384	13 × 13	3 × 3	1	SAME	ReLU
C5	Convolution	384	13 × 13	3 × 3	1	SAME	ReLU
S4	Max Pooling	256	13 × 13	3 × 3	2	VALID	–
C3	Convolution	256	27 × 27	5 × 5	1	SAME	ReLU
S2	Max Pooling	96	27 × 27	3 × 3	2	VALID	–
C1	Convolution	96	55 × 55	11 × 11	4	VALID	ReLU
In	Input	3 (RGB)	227 × 227	–	–	–	–

Hands on Machine Learning. Chapter. 14. Aurelien Geron.

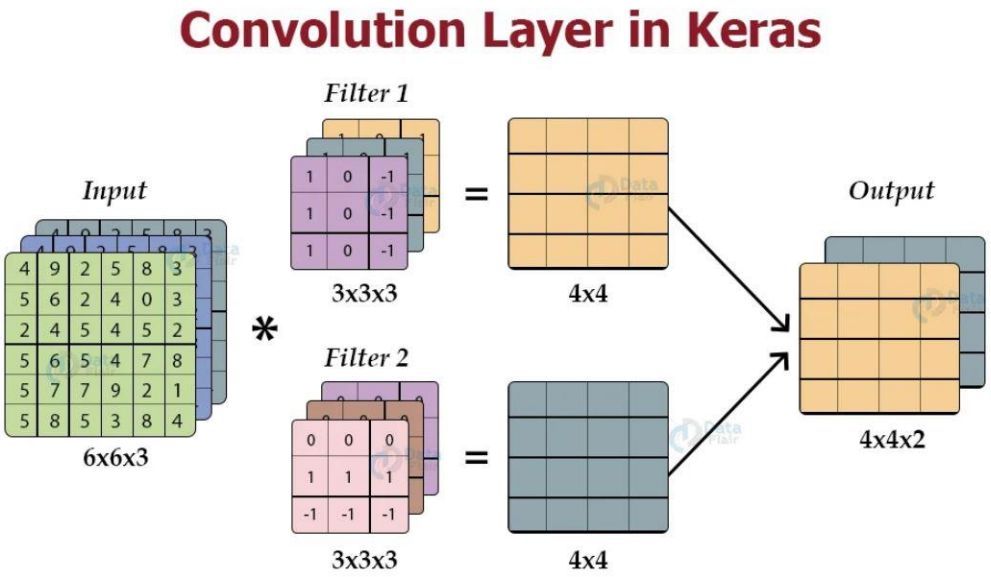
Convolutional Neural Networks



GoogLeNet architecture

Hands on Machine Learning. Chapter. 14. Aurelien Geron.

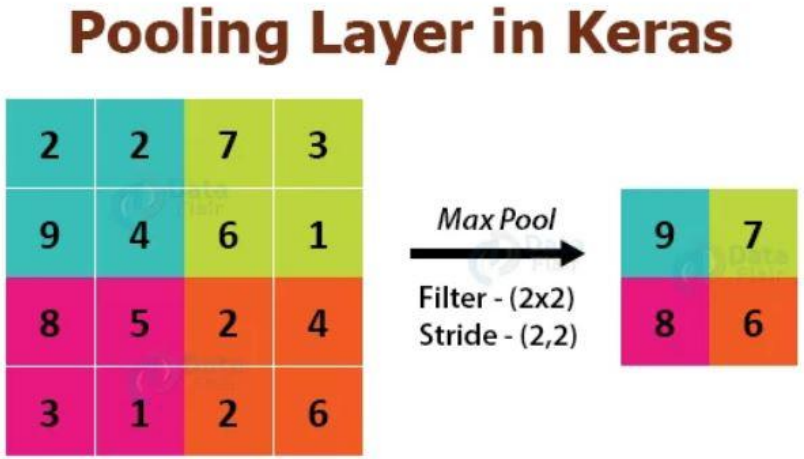
Exercises – Hint. Conv. Layer.



How to visualize conv layers in Keras.

https://data-flair.training/blogs/keras-convolution-neural-network/

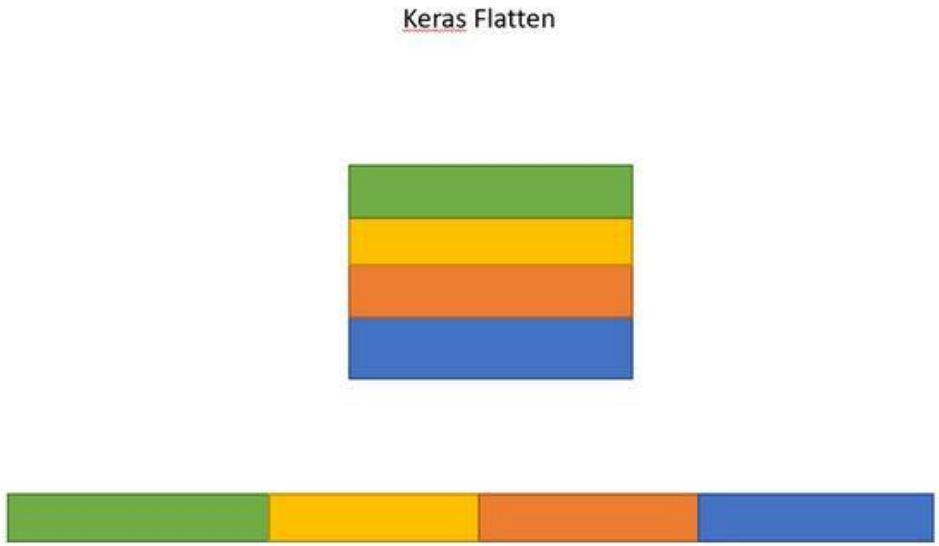
Exercises – Hint. Pooling layer.



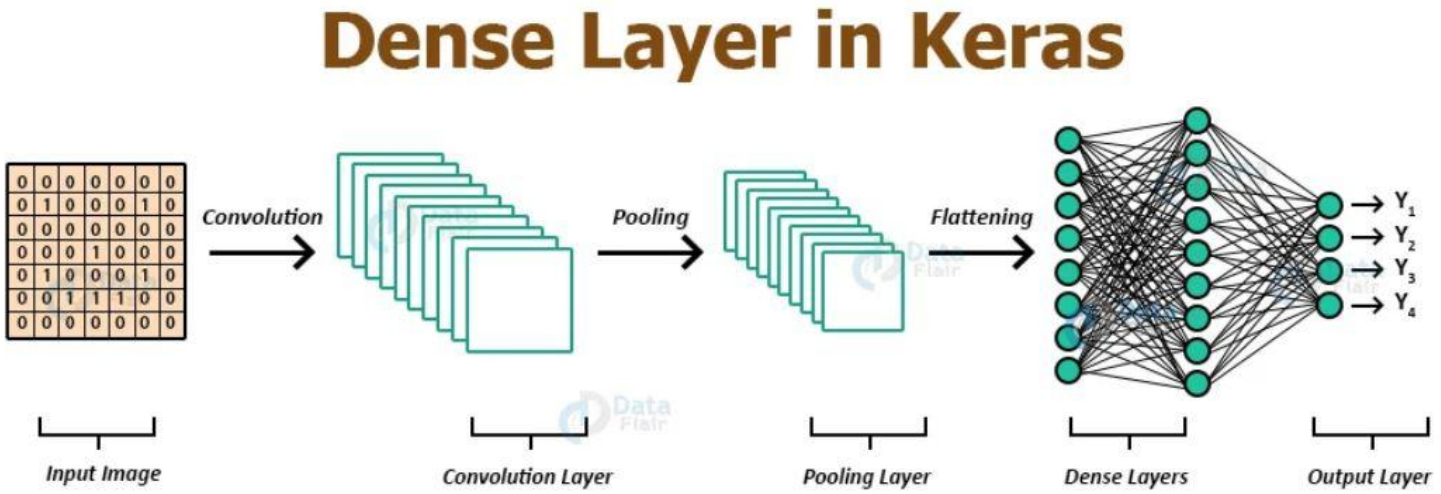
How to visualize pooling layers in Keras.

https://data-flair.training/blogs/keras-convolution-neural-network/

Exercises – Hint. Flatten.



Exercises – Hint. Architecture.



How to visualize pooling layers in Keras.

https://data-flair.training/blogs/keras-convolution-neural-network/

Exercises

Remaining Exercises...

Exercises 4, 5 & 6.

And 7…

Homework

Do the exercises for this week.

Read:
"Hands-On Machine Learning" by Aurelien Geron.
Chapter 14, p. 481 – 500 (v2. p. 445 – 466)
 (Convolutional Neural Networks).

Optional:
Chapter 14. p. 500- 516 (v2. p. 466- 497)
 (Convolutional Neural Networks).
"Hands-On Machine Learning" by Aurelien Geron.